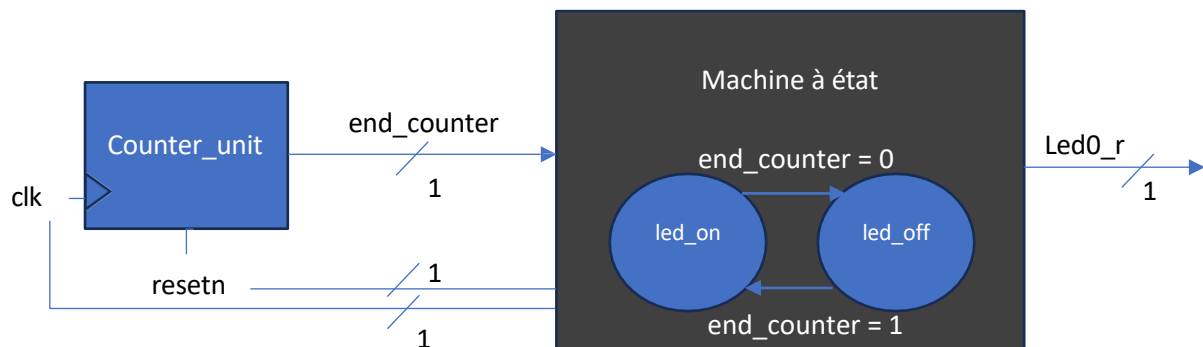
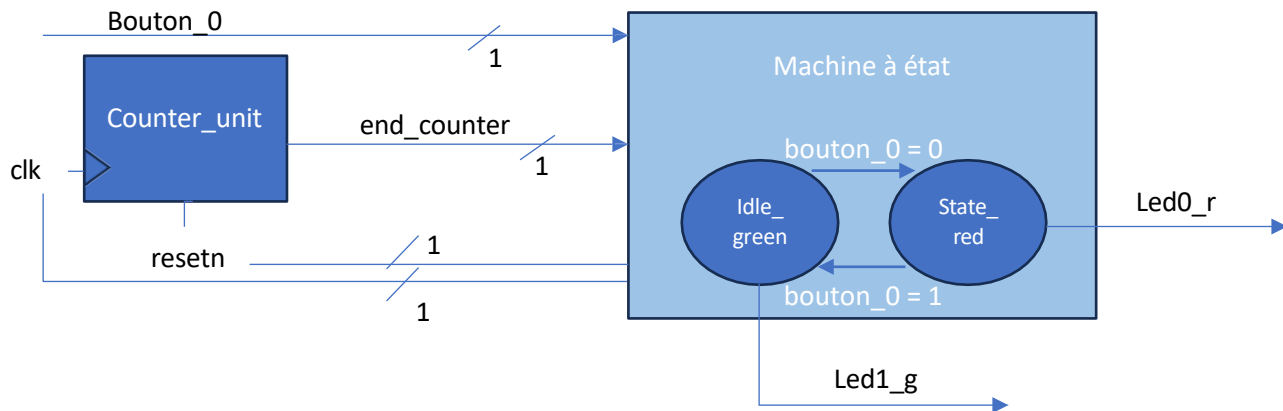


TP4_Pilotage

1. Créez une architecture RTL permettant de faire clignoter une LED (par exemple la led0_r) en utilisant le module Counter_unit du TP2 et une machine à état.



2. Modifiez votre architecture pour piloter une LED rouge et une LED verte. Lorsque le bouton_0 est appuyé, la LED verte est allumée, sinon la LED rouge est allumée.

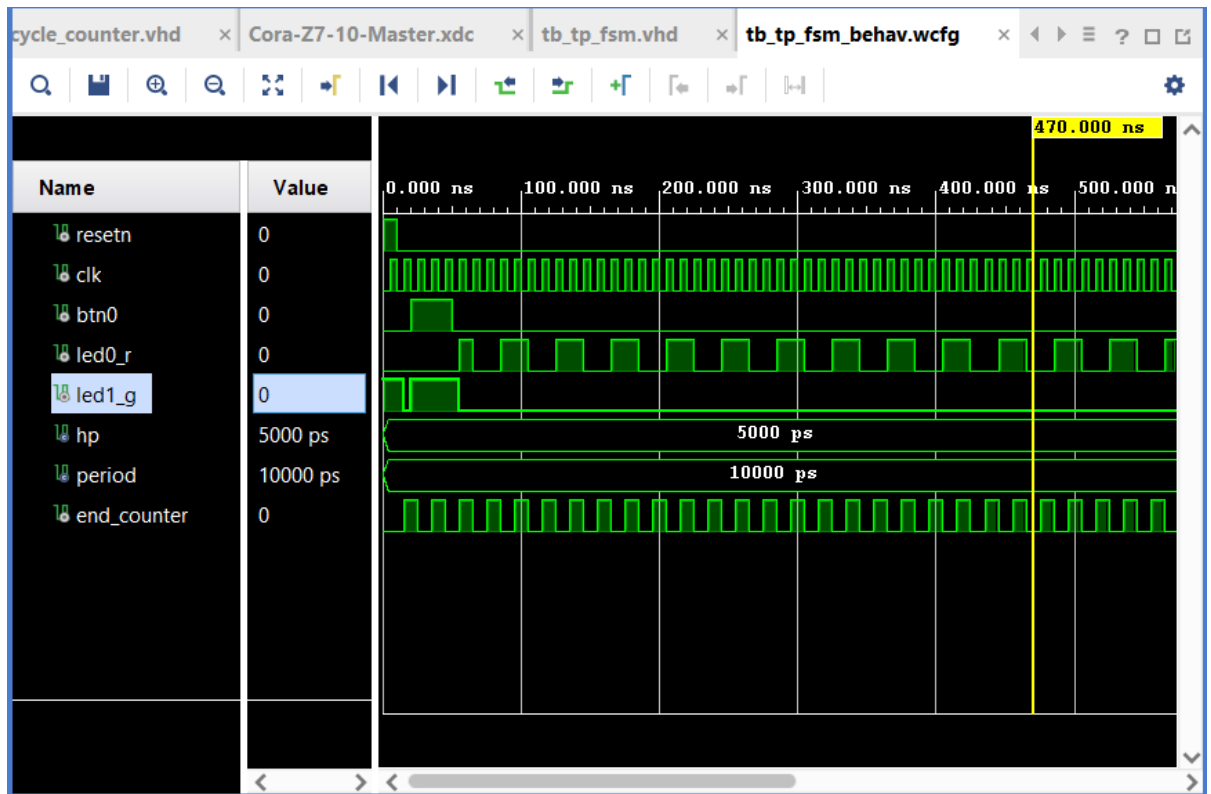


3. Rédigez le code VHDL de votre architecture.

(voir code : counter_unit.vhd, cycle_counter.vhd, tp_fsm.vhd)

4. Réalisez une simulation en rédigeant un testbench. Que se passe-t-il si le bouton est pressé pendant plus d'un cycle d'horloge ?

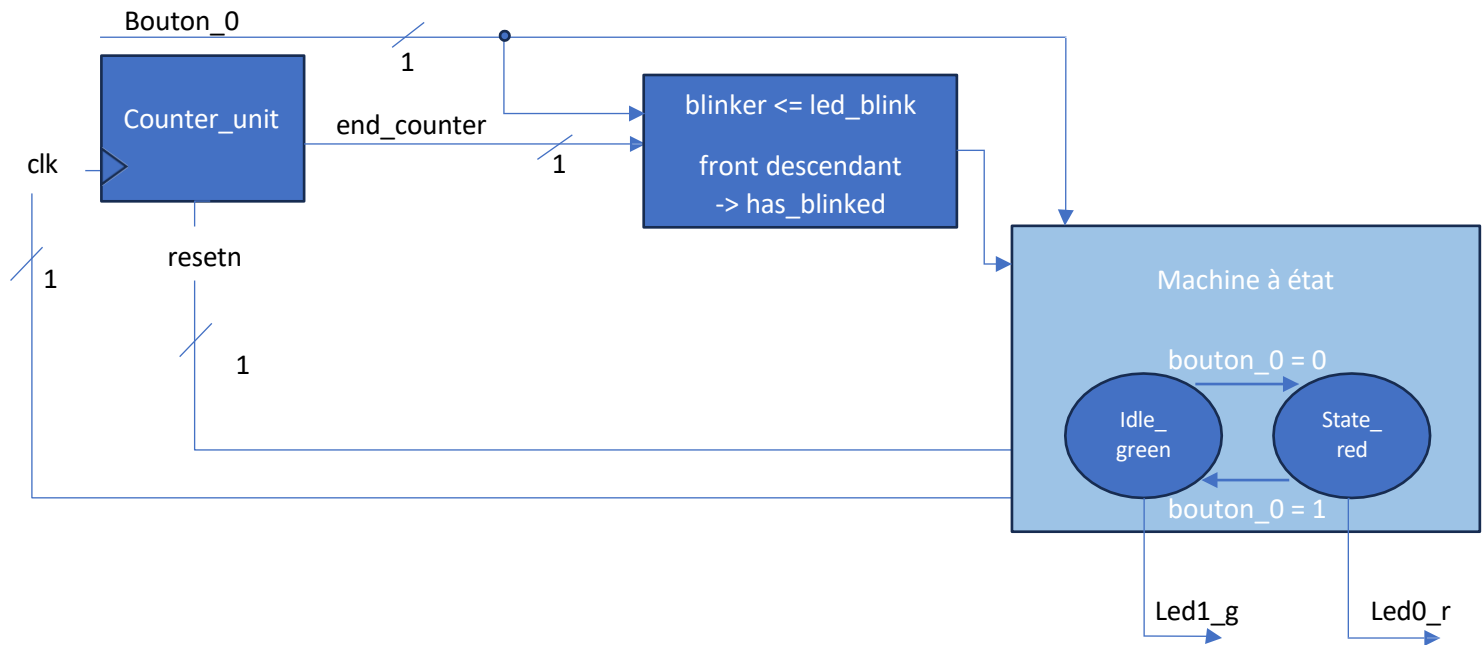
Dans la simulation la led verte reste allumée pendant que le bouton0 est à 1, elle ne clignote pas et ne repasse pas au rouge tant que le bouton0 est enfoncé (voir code tb_tp_fsm.vhd).



5. Que faudrait-il faire pour que la LED ne clignote en vert qu'une seule fois même si le bouton est maintenu ?

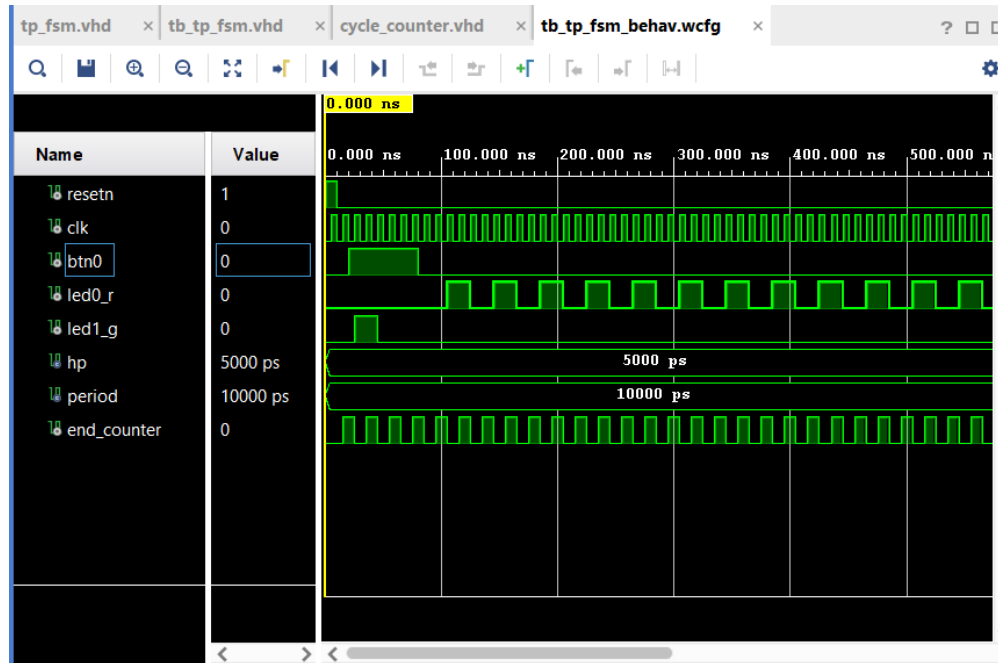
Il faudrait implémenter un signal qui garde en mémoire l'état de led_blink (changement d'état des leds), pour pouvoir détecter quand il change et alors empêcher la led verte de clignoter à nouveau.

6. Ajoutez cette solution à votre architecture RTL.



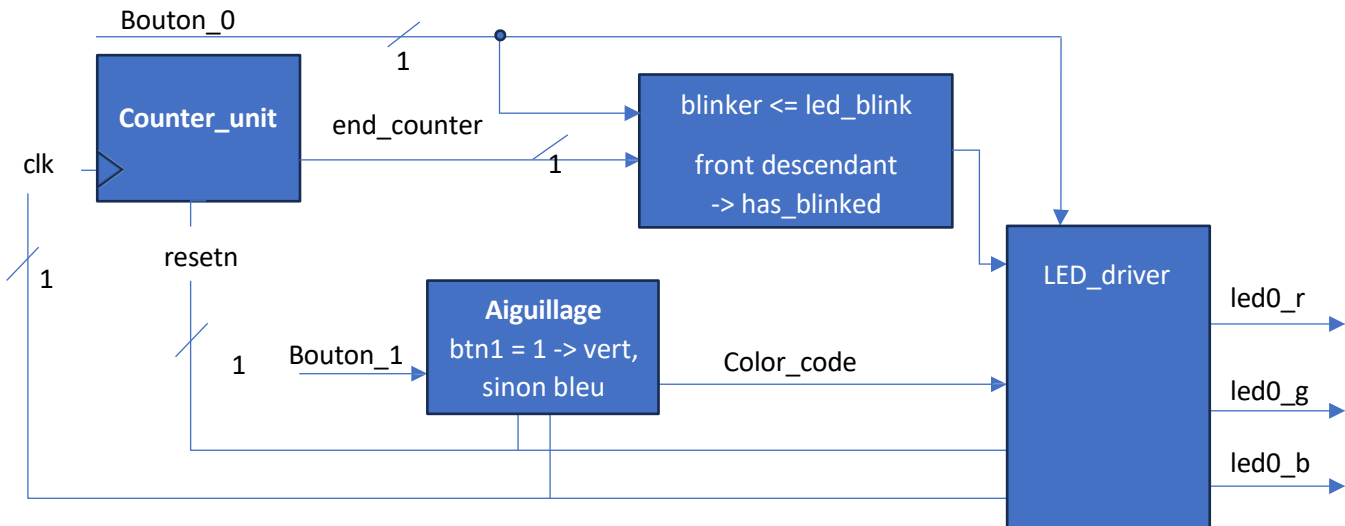
7. Mettez à jour votre code VHDL et vérifiez votre résultat à la simulation.

On observe que la led verte s'allume lorsque le bouton 0 est à 1 mais elle ne reste plus allumée tout du long et s'éteint après 2 périodes comme prévu (voir code tp_fsm.vhd).



8. Créez un module de pilotage d'une LED RGB en RTL. Ce dernier doit permettre de faire clignoter une LED RGB connectée en sortie d'une couleur définie par un code couleur donné en entrée. Le changement de couleur de la LED RGB n'a lieu que si un signal update est reçu.

9. Ajouter la logique nécessaire pour piloter les entrées/sorties de votre module. Le signal update doit recevoir 1 uniquement lorsque le bouton_0 vient d'être appuyé, maintenir le bouton enfoncé ne doit pas maintenir le signal update à 1. Le signal color_code doit recevoir soit le code couleur « vert » si le bouton_1 est pressé, il doit recevoir le code couleur « bleu » sinon. Les LED R, G et B sont connectées avec les couleurs respectives de la LED_0.



10. Ecrivez le code VHDL correspondant à votre architecture.

(voir code : LED_driver.vhd (Q8), pilote_led_driver.vhd (Q9))

11. Ecrivez un testbench pour tester votre design.

(voir code : tb_led_driver.vhd (teste LED_driver seul, les 4 couleurs) + tb_pilote_led_driver.vhd)

12. Vérifier votre résultat à la simulation.

J'ai réalisé deux testbenchs, un pour le module LED_driver de la question 8, qui teste donc tous les aspects de color_code.

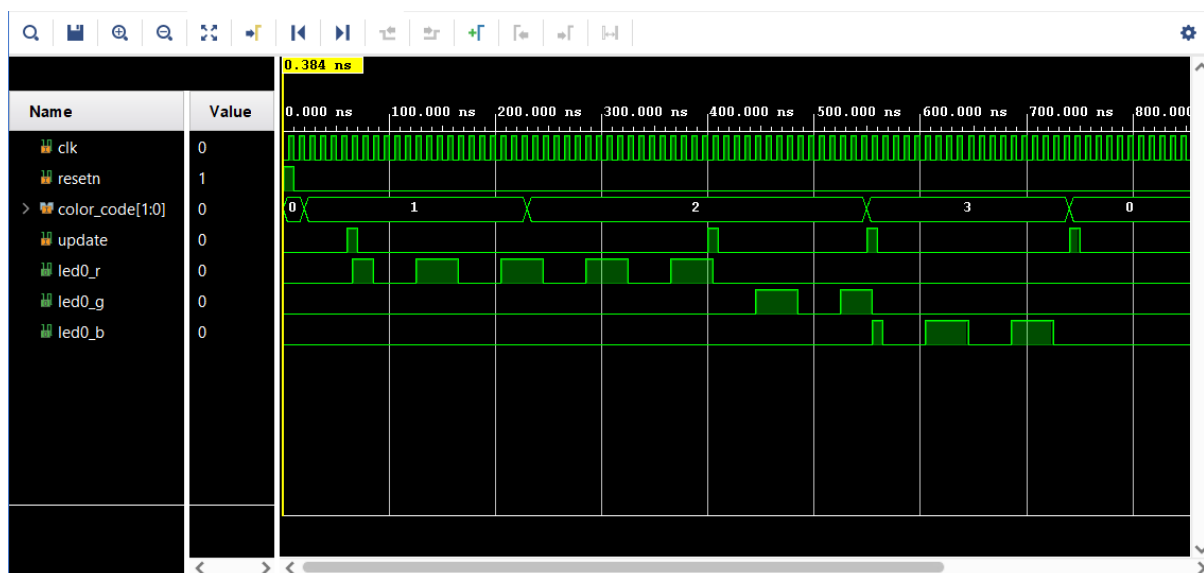
Et un deuxième pour la question 9 qui teste l'implémentation des boutons 0 et 1 avec le changement de couleur qui s'en suit :

Avec ce premier testbench je vérifie que les 4 états de color_code fonctionnent correctement :
Après un reset aucune led n'est allumée, si color_code change mais que update n'est pas incrémenté la couleur ne change pas.

On voit bien qu'après un reset aucune led se s'allume même si color_code passe à 01, tant que update n'ai pas appelé la couleur ne change pas.

Le changement de couleur fonctionne également si on change la valeur de color_code et update à 1 rapidement.

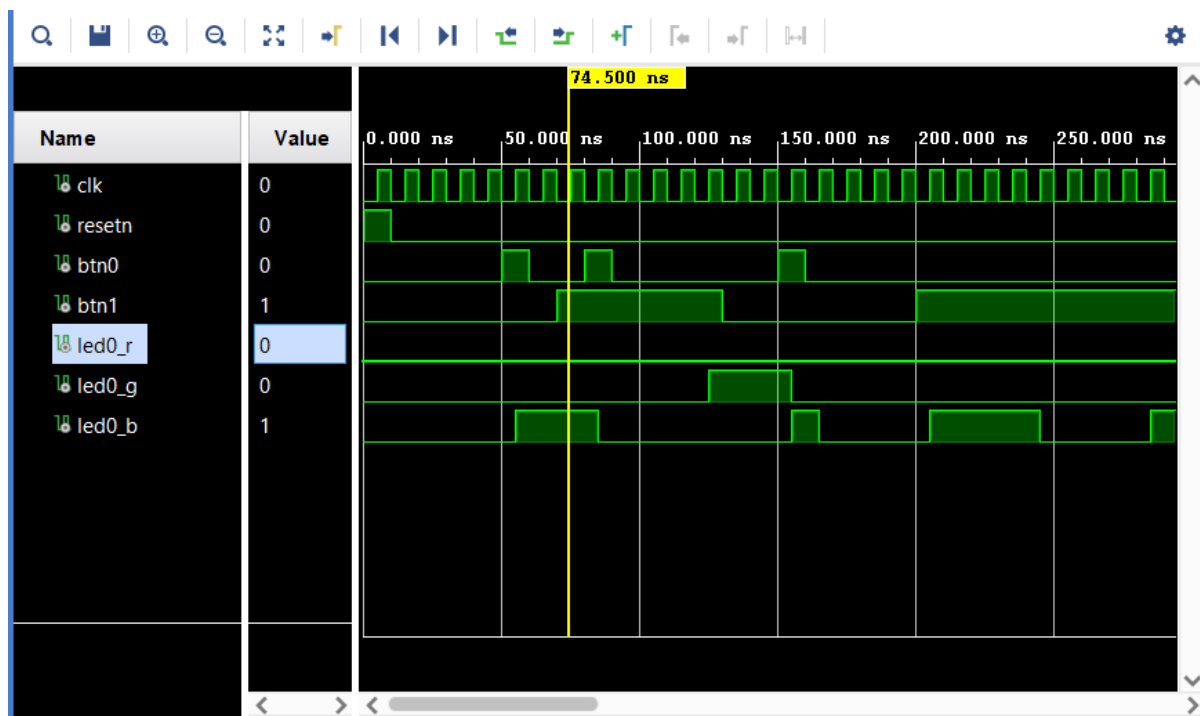
Et l'état 00 de color_code fonctionne correctement.



Avec ce deuxième testbench je test la Q9, l'alternance entre les leds bleues et vertes. On peut voir qu'après un reset les leds restent éteinte si le bouton 0 n'est pas appuyé.

Le bouton 0 démarre le cycle et permet à la led bleue de s'allumer, on voit également que le bouton appuyé seul ne provoque aucun changement comme dit dans l'énoncé. Mais le fait d'avoir le bouton 0 et 1 appuyé provoque le changement sur l'état vert.

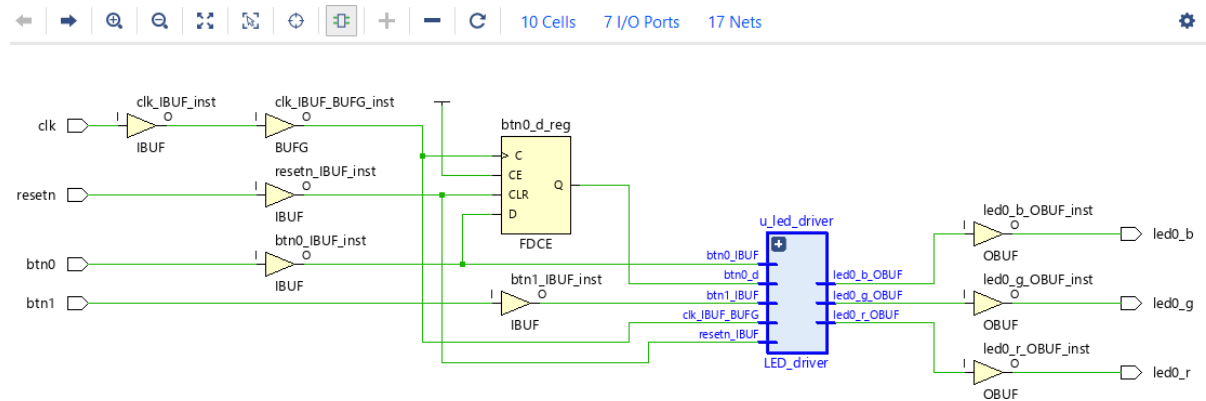
Et si on réappuie sur 0 seul l'état rebascule sur bleu.



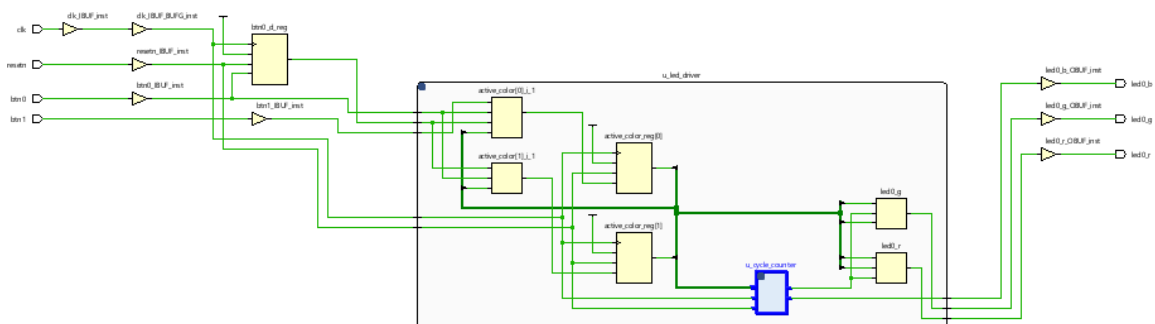
13. Réalisez une synthèse et étudiez le rapport de synthèse, les ressources utilisées doivent correspondre à votre schéma RTL.

Après avoir réalisé la synthèse j'obtiens cette schématique où on retrouve bien les 4 entrées (clk, resetn, btn0 et btn1), ainsi que les 3 sorties des leds (led0_r, led0_g et led0_b).

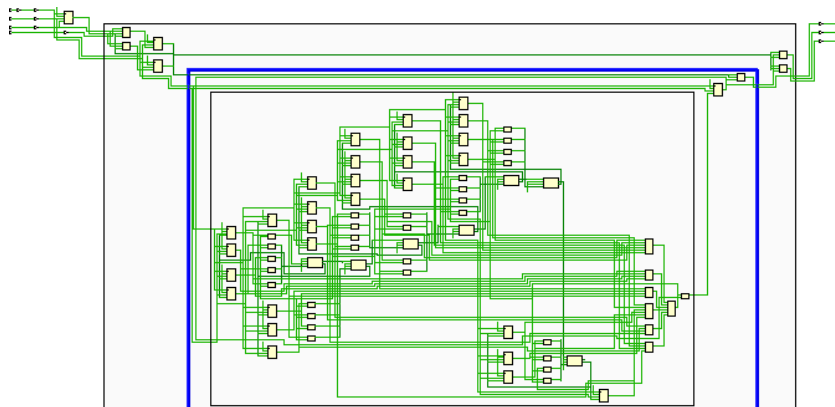
On voit également les modules de btn0 et btn1 qui pilotent le composant u_led_driver.



En plongeant plus loin dans l'architecture, on a bien notre compteur de cycle à l'intérieur du composant u_led_driver ainsi que les 4 valeurs possible de color_code (active_color). Pour ce projet nous avons besoin que des sorties des leds bleue et vertes comme on peut le voir en sortie du composant u_led_driver et u_cycle_counter.



On retrouve l'architecture complète regroupant les modules provoquant le clignotement des leds selon un compteur interne.



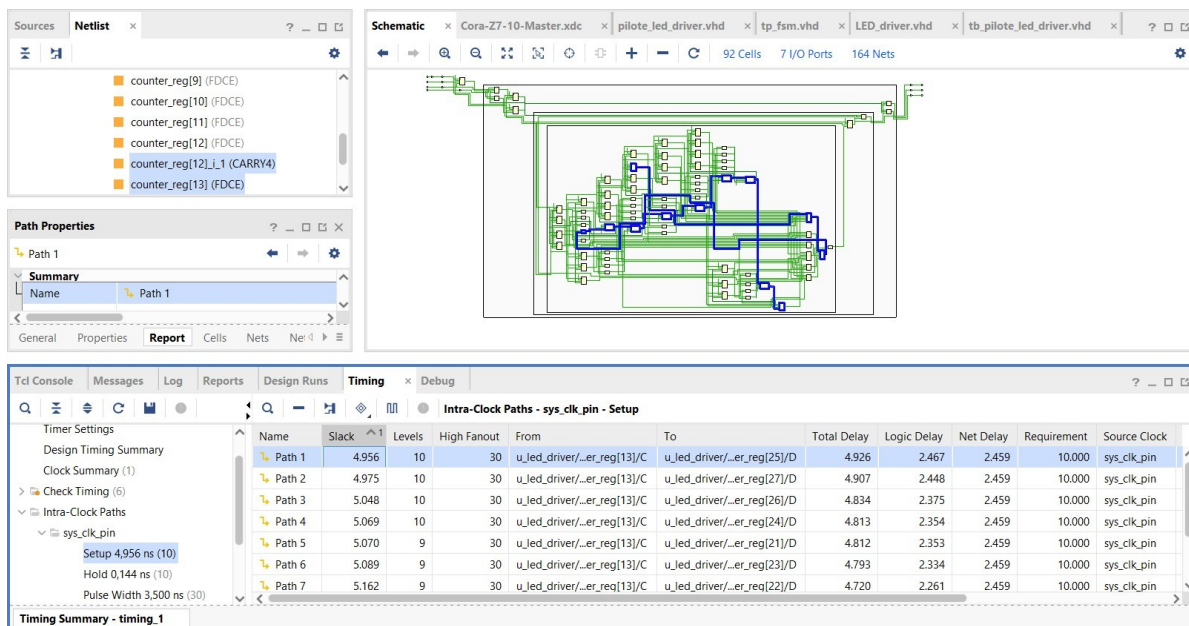
Le design timing summary est bon, le WNS et WHS sont positifs et toutes les contraintes que j'ai spécifiées sont respectées.

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 4,956 ns	Worst Hold Slack (WHS): 0,144 ns	Worst Pulse Width Slack (WPWS): 3,500 ns
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 31	Total Number of Endpoints: 31	Total Number of Endpoints: 33

All user specified timing constraints are met.

De plus les signaux parcourent correctement tout le circuit tout en respectant les délais accordés :



14. Effectuez le placement routage et étudiez les rapports.

Après avoir tenté une première fois de générer le bitstream, je me suis rendu compte que un de mes bouton n'était pas routé dans le fichier de contrainte .xdc.

Mais un problème est survenue lorsque je me suis rendu compte que j'avais 3 entrées "boutons" mais que sur la carte CORA Z7 il n'y avait que 2 boutons pilotables.

J'ai donc passé l'entrée reset sur un port digitale I/O, je l'ai instancié sur PULLUP dans un premier temps mais comme resetn restait à 1 en permanence mon code ne "démarrait" jamais, je l'ai donc mis sur PULLDOWN et tout s'est déroulé comme prévu.

15. Générez le bitstream et vérifiez que vous avez le comportement attendu sur carte.

Suite à la génération du bitstream, j'ai remarqué que mes boutons étaient inversés, je les ai donc échangés dans le fichier .xdc.

J'ai également rencontré un problème, les leds ne clignotaient pas, elles restaient juste allumées. J'ai compris par la suite que c'était un problème de counter_max. Mon compteur comptait jusqu'à 5, c'est beaucoup trop court pour pouvoir constater un clignotement à l'œil nu, je l'ai donc passé à 49 999 999 en faisant quelques tests.

Le comportement de la carte était finalement celui attendu (voir vidéo de démonstration)